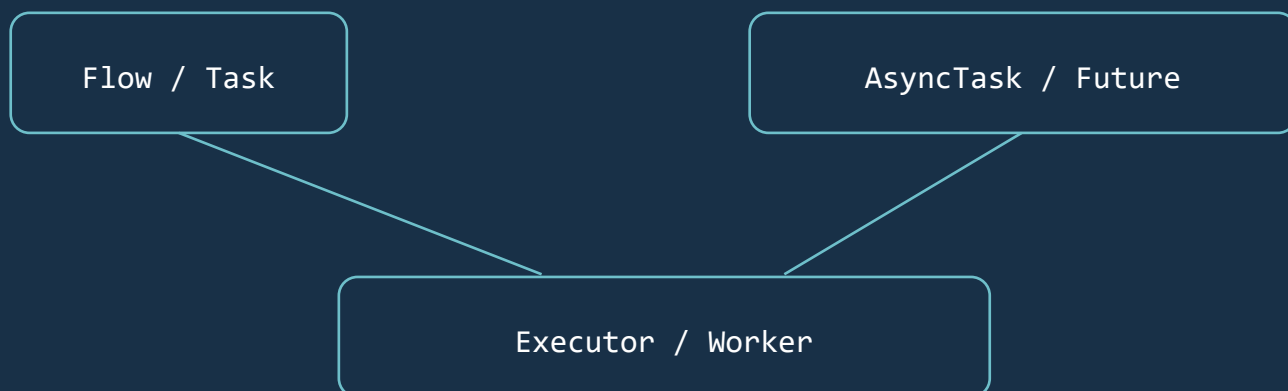


C++20 / HEADER-ONLY / TASK PARALLELISM

TaskflowLite

完整使用手册

从任务图入门到执行器内部机制



入门与集成 / 全部主要组件 / 28 个可执行示例

调度与存储 / 生命周期与异常 / 构建、测试与排错

3.2.0 · 2026-09-22 本地源码快照

BASE 91424ac + WORKING TREE CHANGES | MIT

阅读指南与范围

本手册面向具备 C++ 基础的使用者和维护者。前半部分按实际使用顺序介绍安装、构图、提交、结果、动态任务与执行控制；后半部分解释调度、容器、存储、诊断和验证方式。可先运行第 04 节，再按类型选择指南阅读。

文档约定

- 以当前可执行实现为准。历史注释与实现冲突时，不把注释当作行为保证。
- 公开用法与内部组件分开说明。以下划线开头的方法、Work 状态和队列协议用于理解源码，不建议应用程序直接依赖。
- 除包含 main 的完整程序外，C++ 代码块均为独立函数体片段，默认已有 `tf1::Executor executor(4)`，并包含下列公共头文件。

COMMON HEADERS

```
#include <taskflowlite/taskflowlite.hpp>
#include <taskflowlite/core/scoped_exception_anchor.hpp>
#include <algorithm>
#include <array>
#include <atomic>
#include <cassert>
#include <fstream>
#include <iostream>
#include <memory>
#include <numeric>
#include <stdexcept>
#include <string>
#include <tuple>
#include <vector>
```

如何阅读示例

代码中的 `assert` 表示预期结果；若自行以定义了 `NDEBUG` 的配置编译，标准 `assert` 会关闭。手册验证过程额外保留结果检查。不要把示例中的任务数量、线程数或计数当作性能建议。

手册覆盖当前主要公开组件及其协作关系，并给出全部示例的索引。它不是每个模板重载的逐字 API 转录；精确约束、返回类型和 `noexcept` 条件可在对应头文件或生成的 Doxygen 文档中查阅。

来源：taskflowlite/taskflowlite.hpp；README.md；examples/；cmake/。

目录 / 01

从公开用法到内部实现；点击条目可直接跳转。

01 / 库的定位与整体模型	5
02 / 选对类型与所有权	6
03 / 环境、安装与 CMake 集成	7
04 / 第一个完整程序	8
05 / Flow、FlowBuilder 与依赖图	9
06 / Task、TaskView 与任务编辑	10
07 / Callable 协议、批量构造与参数	11
08 / 重复执行与 Module 组合	12
09 / Executor 与提交入口	13
10 / AsyncFuture 与结果类型	14
11 / AsyncTask 与延迟启动	15
12 / 动态依赖与引用保留	16
13 / Runtime 与协作等待	17
14 / TaskGroup 与局部恢复	18
15 / SubFlow 动态子图	19
16 / Branch 与 MultiBranch	20
17 / Jump、MultiJump 与循环	21
18 / TaskObject 与 AsyncTaskObject	22
19 / Semaphore 与资源配额	23
20 / 异常传播与等待接口	24

目录 / 02

从公开用法到内部实现；点击条目可直接跳转。

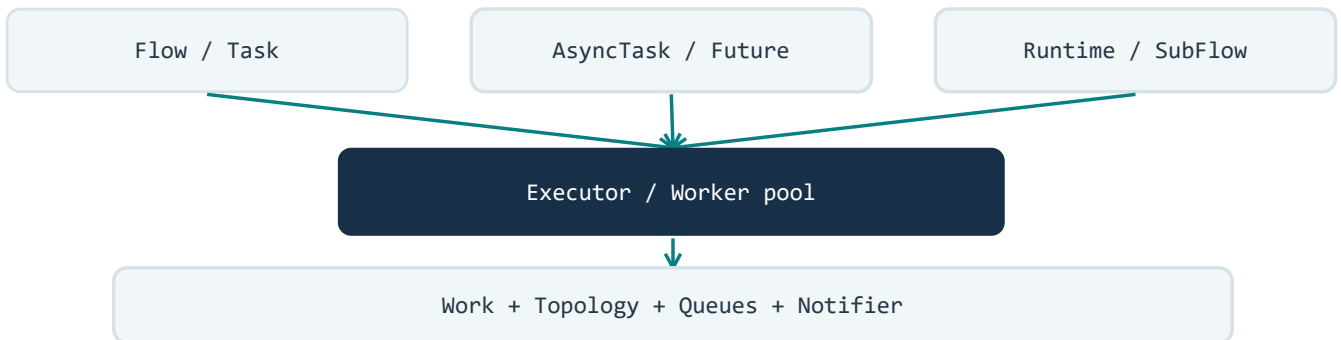
21 / ScopedExceptionAnchor	25
22 / 协作取消与停止请求	26
23 / TaskObserver 任务观察者	27
24 / Worker、WorkerHandler 与 Context	28
25 / 图导出、命名与 D2	29
26 / 常见并行模式	30
27 / 完整案例：受限资源的并行批处理	31
28 / 内部调度：从提交到完成	32
29 / Work、Graph、Topology 与 ResultSlot	33
30 / BoundedQueue 与 SharedWorkStack	34
31 / Notifier、SpinMutex 与随机选择	35
32 / SmallVector、ObjectPool 与存储	36
33 / Concepts、工具类型、枚举与宏	37
34 / 构建选项、依赖与预设	38
35 / 测试、CI 与文档生成	39
36 / 常见问题与当前边界	40
37 / 性能测量与优化方法	41
38 / 组件与源码速查	42
39 / 全部示例阅读路线	43
40 / 来源、验证记录与后续维护	44

01 / 库的定位与整体模型

TaskflowLite 是仅头文件的 C++20 任务并行库。用户定义“做什么”和“谁先于谁”，Executor 在线程池中执行已经满足依赖的任务。库同时支持预先构建的任务图和运行时提交的异步任务。

三个层次

层次	核心组件	负责的问题
描述工作	Flow、Task、对象任务	保存 callable、配置依赖与资源
提交与控制	Executor、AsyncFuture、Runtime、TaskGroup	何时启动、何时完成、如何取得结果
内部执行	Work、Topology、Worker、队列与通知器	就绪判定、工作窃取、等待唤醒和回收



普通图通过依赖边表达顺序；Branch 选择条件路径；Jump 显式激活跳转目标，使重试或循环可以在受控拓扑中实现。图形看起来有环，不意味着可以给普通任务随意连成环。

适合怎样的工作

适合计算依赖、分批处理、并行归约、阶段式工作流和执行中产生子任务的场景。任务本身仍是普通 C++ 代码；共享数据、外部资源和阻塞调用的行为由应用负责。此库不提供分布式调度、进程隔离或自动 I/O 异步化。

入口头文件为 taskflowlite/taskflowlite.hpp，命名空间为 tf1。可从 tf1::version 或 TASKFLOWLITE_VERSION_* 查询源码版本。项目采用 MIT 许可证。

来源：taskflowlite/taskflowlite.hpp；core/forward.hpp；core/enums.hpp；LICENSE。

02 / 选对类型与所有权

需求	建议入口	所有权和使用边界
已知任务依赖	Flow、Task	Flow 拥有节点；Task 借用节点
立即计算并取结果	Executor::async	返回共享 AsyncFuture<R>
先配置后启动	Executor::defer_async	返回 Idle 状态的 AsyncTask<R>
运行中派发子工作	Runtime	回调内上下文，不可逃逸
动态构造子图	SubFlow	构图后显式 run，等待子图完成
作用域内分组等待	TaskGroup	绑定当前任务，等待组内工作
将对象放进任务	TaskObject / AsyncTaskObject	object 访问业务状态
限制资源并发	Semaphore	资源对象须比相关执行活得更久
观察任务和线程	TaskObserver / WorkerHandler	分别观察任务调用和线程生命周期

两种句柄不要混淆

Task 和 TaskView 是静态图节点的非拥有句柄。复制它们不会复制节点，也不会延长节点寿命。Flow::erase 或 clear 后，受影响的句柄不能继续使用。TaskView 用于只读查询，不等于冻结快照。

AsyncTask 和 AsyncFuture 是异步 Work 的共享句柄。复制句柄共享同一次执行和结果；释放最后一个句柄不等于强制终止任务。任务运行引用和依赖持有的引用也参与底层对象的存活。

生命周期基本顺序

先准备数据与资源，再建图或配置任务，随后提交，最后等待并读取结果。以左值提交的图、按引用捕获的数据、外部业务对象和信号量必须在整个执行期间有效。Executor 必须在创建的任务启动和执行期间有效。

同一个句柄对象与 reset、赋值、移动或析构不能并发操作；不同句柄共享任务并不自动使业务状态线程安全。

来源：core/task.hpp；core/async_future.hpp；core/async_task.hpp；core/context.hpp。

03 / 环境、安装与 CMake 集成

需要 C++20 编译器及支持 `std::format`、原子等待等功能的标准库。GCC 使用 GCC 13 或更新版本的配套 `libstdc++`；Clang 也需要匹配的标准库。CMake 集成要求 3.21 或更新版本。

作为子项目使用

将仓库放在 `external/taskflowlite`，应用代码保存为 `main.cpp`：

CMAKE

```
cmake_minimum_required(VERSION 3.21)
project(my_app LANGUAGES CXX)
add_subdirectory(external/taskflowlite)
add_executable(my_app main.cpp)
target_link_libraries(my_app PRIVATE
    TaskflowLite::taskflowlite)
```

COMMAND

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --config Release --parallel 4
```

安装后由其他项目使用

在库仓库根目录配置纯安装项目，不需要先编译静态库：

COMMAND

```
cmake -S . -B build/install -DTFL_BUILD_EXAMPLES=OFF
cmake --install build/install --prefix ./install
```

消费项目把 `add_subdirectory` 替换为 `find_package(TaskflowLite CONFIG REQUIRED)`，仍链接相同目标。配置时通过 `CMAKE_PREFIX_PATH` 指向安装目录的绝对路径。

导出目标传递头文件路径、C++20 和平台链接要求。直接复制 `taskflowlite/` 目录也可使用，但须自己配置线程、编译选项和适用平台的原子库链接。核心库无需第三方运行时；Catch2、Taskflow 对比程序和 Doxygen 是可选开发依赖。

来源：CMakeLists.txt；cmake/TaskflowLiteConfig.cmake.in；cmake/README.md。

04 / 第一个完整程序

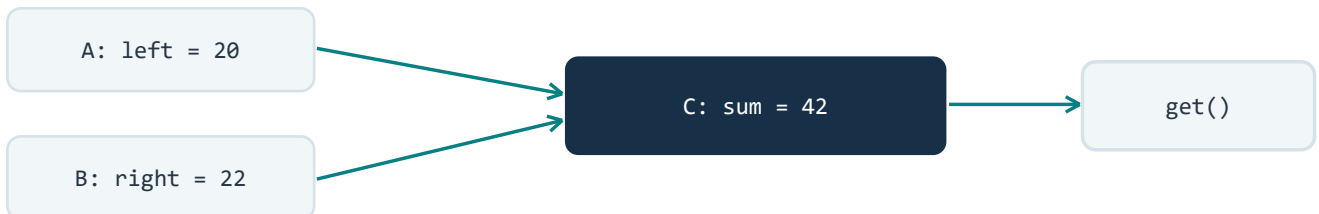
两个计算任务可以并行，一个汇总任务等待两者完成。这个例子同时展示 Flow 的所有权、Task 建边、Executor 提交和 Future 同步。

```
C++20
#include <taskflowlite/taskflowlite.hpp>
#include <iostream>

int main() {
    tf1::Executor executor(4);
    tf1::Flow flow;
    int left = 0, right = 0, result = 0;

    auto a = flow.emplace([&] { left = 20; });
    auto b = flow.emplace([&] { right = 22; });
    auto c = flow.emplace([&] {
        result = left + right;
    });
    a.name("left");
    b.name("right");
    c.name("sum");
    c.succeed(a, b);

    executor.async(flow).get();
    std::cout << result << '\n';
    return result == 42 ? 0 : 1;
}
```



输出为 42。a 和 b 写入不同变量；c 在依赖完成后读取它们；主线程在 get 返回后读取 result。这里的安全性来自明确的依赖和完成同步，不是因为普通 int 自动具备跨线程同步能力。

如果只调用 async 后立即离开作用域，数据和 Flow 可能先被销毁。实践中优先把“提交并等待”的边界写清楚，再逐步扩大并发范围。

来源：examples/01_basic_dag.cpp；core/executor.hpp；core/flow.hpp。

05 / Flow、FlowBuilder 与依赖图

Flow 独占 Graph 和节点，不能复制；移动会转移所有权。FlowBuilder 提供构图操作，Flow 和 SubFlow 复用这套接口。构图不意味着执行，只有提交后执行器才会调度节点。

操作	用途
emplace(callable)	添加任务，返回 Task
placeholder()	添加无业务 callable 的依赖节点
a.precede(b) / b.succeed(a)	建立同一图中的有向边
linearize(a, b, c)	将多个任务连成串行链
size / empty / for_each	查询或遍历图节点
erase / clear	静止状态下删除节点或清空图

C++20

```
tfl::Flow flow;
int step = 0;
auto begin = flow.placeholder();
auto a = flow.emplace([&] { step = 1; });
auto b = flow.emplace([&] { step *= 2; });
flow.linearize(begin, a, b);
executor.async(flow).get();
assert(step == 2);
```

默认校验

默认建边检查空目标、不同图、重复边、普通节点自环和不经过 Jump 的严格循环。默认校验是构图逻辑检查，不替代运行期间的数据同步。不要把同一图并发提交，或在运行中移动、清空、修改图。

precede<false>、succeed<false> 和 linearize<false> 是跳过校验的显式入口，调用者承担拓扑合法性责任。只有在图生成逻辑经过验证、并且构图成本确实重要时才考虑使用。具体重载见示例 33。

不同、互不共享可变图状态的 Flow 可以交给同一个 Executor 执行。同一 Flow 完成后可再次提交，其 callable 中保留的业务状态不会自动清零。

来源：core/flow.hpp; core/flow_builder.hpp; examples/33_unchecked_task_links.cpp。

06 / Task、TaskView 与任务编辑

Task 是配置图节点的句柄，TaskView 提供只读观察入口。句柄相等和 hash_value 按节点身份判定，不按 callable 内容或名称判定。复制 Task 后，两份句柄仍操作同一个节点。

接口类别	常用操作
身份和元数据	valid、name、type、hash_value
边关系	num_predecessors、num_successors、for_each_predecessor、for_each_successor
配置清理	remove_predecessor、remove_successor、clear_predecessors、clear_successors
工作替换	work(callable)，保留节点和边
资源与观察	acquire、release、register_observer、unregister_observer

```
C++20
tfl::Flow flow;
int value = 0;
auto task = flow.placeholder();
task.name("editable");
task.work([&] { value = 10; });
executor.async(flow).get();

task.work([&] { value += 32; });
executor.async(flow).get();
assert(value == 42);
```

编辑和查询的边界

编辑只应发生在图未执行时。名称返回的 string_view 也受后续改名、移动和销毁影响。删除边可能改变遍历位置；不要把已保存的后继索引当作跨编辑稳定标识。

当前 work 替换路径先销毁旧载荷，再构造新 callable。若新构造抛异常，不能假设旧 callable 仍可执行；应重新安装有效 callable 或重建图。手册没有将这个接口描述为强异常保证。

TaskView 的 has_exception_ptr / exception_ptr 查询本节点当前持有的异常指针。静态图异常可能已经移交提交 Future；节点上查询不到指针，不代表 Future::get 不会抛异常。应在同步后查询。

来源：core/task.hpp；core/work_storage.hpp；examples/28_task_editing.cpp。

07 / Callable 协议、批量构造与参数

库通过 C++ concepts 区分 callable 协议。普通无参 callable 对应 Basic；接受 Runtime&、SubFlow&、Branch&、MultiBranch&、Jump& 或 MultiJump& 的 callable 获得对应执行语义。这些参数由框架在执行时注入。

常用写法

```
C++20
tfl::Flow flow;
int first = 0, second = 0;
auto [a, b] = flow.emplace(
    [&] { first = 20; },
    [&] { second = 22; }
);
auto sum = flow.emplace([&] {
    first += second;
});
sum.succeed(a, b);
executor.async(flow).get();
assert(first == 42);
```

批量 `emplace` 返回可用于结构化绑定的任务集合。FlowBuilder 还支持 `tuple` 与 `tfl::pack` 组织批量输入；模块可附带次数或停止谓词。复杂重载组合见示例 11，不应仅凭实参个数猜测选中的协议。

参数和捕获

对普通用法，lambda 捕获最直观。按值捕获适合独立状态；移动捕获适合所有权转移；引用捕获要求目标对象存活至任务完成。`std::ref` / `std::cref` 表示借用，也不延长对象寿命。

`Executor::async` 的 callable 后面可以是异步依赖句柄，不是任意业务参数。业务参数应捕获或预先绑定，避免把“函数实参”和“前驱依赖”混为一谈。

静态图节点通常通过引用或对象状态输出结果；需要带类型的结果通道时使用 `AsyncFuture<R>`。Runtime / SubFlow 类型的异步 callable 也可以返回结果，但访问结果前仍须完成相应子工作。

来源：core/traits.hpp; core/flow_builder.hpp; examples/11_flow_emplace.cpp。

08 / 重复执行与 Module 组合

重复执行复用图结构；每轮运行的是同一组节点。应用数据是否复位由 callable 或外围代码决定。次数为 0 时不会执行图体，停止谓词在每轮开始前检查，返回 true 表示结束。

```
C++20
tfl::Flow flow;
int count = 0;
(void)flow.emplace([&] { ++count; });
executor.async(flow, SULL).get();
assert(count == 5);

executor.async(flow, [&]() noexcept {
    return count >= 10;
}).get();
assert(count == 10);
```

静态嵌套模块

将一个 graph_holder 放入另一个图，得到负责执行子图的 Module 节点。模块也可配置执行次数或终止谓词。静态嵌套与 SubFlow 的区别在于：模块通常提前构图，SubFlow 在当前任务执行期间构图。

```
C++20
tfl::Flow child, parent;
int visits = 0;
(void)child.emplace([&] { ++visits; });
auto module = parent.emplace(child, 3ULL);
module.name("three_passes");
executor.async(parent).get();
assert(visits == 3);
```

以左值装入的子图是借用关系，必须活得足够久；以右值传入可转移图所有权。无论采用哪种形式，不要让同一底层图同时被多个执行路径使用。graph_holder 允许自定义持图对象，业务对象模块的构造方式见第 18 节及示例 31、32。

来源：core/flow_builder.hpp; core/work_invokers.hpp; examples/03_loop.cpp; examples/08_subflow.cpp。

09 / Executor 与提交入口

Executor 拥有工作线程和调度状态。应用通常长时间复用一個執行器，而不是為每個小任務反復創建線程池。構造時顯式指定正數線程數，便於重現實驗與控制資源。

入口	返回或行為
<code>async(callable, dependencies...)</code>	立即提交，返回 <code>AsyncFuture<R></code>
<code>async(graph, ...)</code>	提交一次、固定次數或謂詞控制的圖執行
<code>defer_async(...)</code>	創建綁定執行器的 <code>Idle AsyncTask<R></code>
<code>silent_async(...)</code>	立即提交，無結果句柄
<code>wait_for_all()</code>	等待執行器已納入管理的工作完成
析構	等待所管理工作並結束工作線程

```
C++20
std::atomic<int> completed{0};
for (int i = 0; i < 8; ++i) {
    executor.silent_async([&] {
        completed.fetch_add(1, std::memory_order_relaxed);
    });
}
executor.wait_for_all();
assert(completed.load() == 8);
```

等待範圍

尚未 start 的延遲任務沒有提交，不應期待 `wait_for_all` 會替你啟動它。其他線程持續提交新工作時，業務上仍需定義清楚“收工”邊界，不能把一次空閒觀察當作永久沒有任務。

`silent_async` 適合已經在任務內部處理錯誤的工作。頂層無結果任務沒有可供調用方 `get` 的錯誤通道。圖執行完成回調也不應替代 `Future` 的結果與異常同步。

執行器的 `worker`、拓撲計數等查詢用於診斷；隊列大小和空閒快照不能替代完成同步。Worker 內部不要通過等待整個執行器的方式等待包含自身的工作。

來源：`core/executor.hpp`；`examples/02_parallel.cpp`；`examples/14_async_task_chain.cpp`。

10 / AsyncFuture 与结果类型

`AsyncFuture<R>` 表示同一次异步执行的共享状态。它可复制、移动、reset，并可重复调用 `get`。与“取走一次值”的使用模型不同，`get` 不消费结果。

R	get 的行为	生命周期重点
值类型	返回 const R&	引用依赖底层任务继续存活
左值引用	返回原始引用	原始业务对象仍须有效
void	等待并传播异常	没有值可读取

C++20

```
auto value = executor.async([] { return 42; });
auto shared = value;
assert(value.get() == 42);
assert(shared.get() == 42);

auto owner = executor.async([] {
    return std::make_unique<int>(7);
});
const auto& pointer = owner.get();
assert(*pointer == 7);
```

同步和错误

`wait` 只等待，不重抛任务异常；`get` 先等待，再重抛保存的异常，成功时访问结果。`done` / `running` 等状态查询便于观察，不负责延长外部引用数据的寿命。空 `Future` 调用 `get` 会抛出 `tfl::Exception`。

当保存 `get` 返回的引用时，应同时保留一份拥有底层 `Work` 的句柄。尤其不要先从临时 `Future` 取得引用，再让最后的拥有句柄立即销毁。引用结果即使由 `Future` 持有，也不会自动拥有被引用对象。

当前 `ResultSlot` 对不同值类型采用直接或可选存储；部分类型的默认构造可能在提交准备阶段发生并抛出。任务构造失败与任务执行中抛异常属于不同时间点。

来源：core/async_future.hpp; core/result_slot.hpp; examples/29_async_future_results.cpp。

11 / AsyncTask 与延迟启动

`defer_async` 创建绑定当前 `Executor` 的 `Idle` 任务。它允许启动前设置名称、信号量和观察者，或准备依赖关系。创建和转为 `Future` 都不会自动执行。

```
C++20
auto first = executor.defer_async([] { return 21; });
auto second = executor.defer_async([first] {
    return first.get() * 2;
});
first.name("first");
second.name("second");
first.start();
second.start(first);
assert(second.get() == 42);
```



`Idle` 不会因转换成 `Future` 而自动执行；完成后仍可读取共享结果。

启动规则

成功启动只允许一次。前驱必须已经启动或完成，空前驱忽略；空任务、自依赖、`Idle` 前驱和重复启动会被拒绝。对参数校验失败，可以修正后重新启动；不要把这项保证扩大为内存分配失败后的可重试保证。

`start` 始终返回 `AsyncTask&`。`auto task = executor.defer_async(...).start()` 会按值保存共享句柄；不要用 `auto&` 借用该临时任务返回的引用，否则临时对象销毁后引用失效。

`defer_async().start()` 是独立顶层任务，即便调用位置在 `Runtime` 回调内部。需要自动纳入父任务完成计数时，应使用当前 `Runtime` 或 `TaskGroup` 的 `async` / `silent_async`。

来源：core/async_task.hpp；documentation/async-task-dependency-design.md。

12 / 动态依赖与引用保留

异步依赖把“后继可执行”的条件定义为“所有前驱已完成”。它不会自动把结果或异常复制进后继。前驱失败也可能解除完成依赖，因此需要由后继显式 `get` 来传播错误。

C++20

```
auto a = executor.async([] { return 20; });
auto b = executor.async([] { return 22; });
auto sum = executor.async([a, b] {
    return a.get() + b.get();
}, a, b);
assert(sum.get() == 42);
```

情形	当前行为
AsyncTask / AsyncFuture 混合前驱	接受不同结果类型的句柄
已完成前驱	直接满足对应完成条件
重复前驱	不去重，每项分别持有引用和完成计数
空句柄	忽略
右值传入前驱	不移动或清空该句柄
不同 Executor 的已启动任务	可建立依赖，各执行器须保持有效

完成不等于立即释放依赖

后继的 `Work` 持有前驱强引用，保留到后继 `Work` 真正销毁。`get`、完成或销毁某一份句柄都不保证立即释放整条依赖链；还有其他副本时，业务对象仍可能存活。

框架用一个额外的提交保护计数，避免前驱完成过快、而依赖登记尚未结束时提前运行后继。这个并发协议是内部实现，不要求应用操作计数器。

长依赖链由内部迭代释放逻辑回收，避免销毁递归不断增长。需要控制峰值内存时，及时释放不再需要的结果句柄，并避免无意中把整个历史依赖链长期留在容器里。

来源：`core/executor.hpp::link_predecessors`；`core/work.hpp::destroy_async`；依赖说明。

13 / Runtime 与协作等待

Runtime 是当前任务的执行上下文，用于运行中派发子任务。框架注入 Runtime&，它绑定当前 Work、Worker 和 Executor；不能保存到回调之外，也不能交给另一线程使用。

```
C++20
auto parent = executor.async([](tfl::Runtime& rt) {
    auto a = rt.async([] { return 20; });
    auto b = rt.async([a] {
        return a.get() + 22;
    }, a);
    rt.wait();
    return b.get();
});
assert(parent.get() == 42);
```

协作等待做什么

普通 Future::wait / get 会阻塞调用线程。Runtime 的等待路径允许当前 Worker 帮助执行可运行的工作，避免父任务占住唯一工作线程、子任务却永远无法运行的常见问题。

Runtime 还提供图提交、corun 和基于谓词的 wait_until 等入口。等待独立异步任务时，可先用 rt.wait_until([&] { return task.done(); }) 协作推进，再访问其 get；谓词应简短，且使用正确同步读取共享状态。

Runtime 的子任务会计入父任务的完成管理。但如果子任务引用父 callable 的局部变量，必须在这些局部变量离开作用域前显式等待，不能仅依赖框架在 callable 返回后的收尾等待。

默认 async / silent_async 的 InheritTopology 为 false。它控制停止请求父链，不意味着没有父子完成计数，也不能用它推断异常完全隔离。

来源：core/runtime.hpp; core/context.hpp; examples/04_runtime.cpp; examples/24_recursive_runtime.cpp。

14 / TaskGroup 与局部恢复

TaskGroup 在 Runtime 等执行上下文里管理一组子任务，适合给并行子步骤设置清晰作用域。它不是跨线程共享的外部任务容器，应在创建它的 Worker 和任务回调中使用、等待并销毁。

```
C++20
auto parent = executor.async([](tf1::Runtime& rt) {
    tf1::TaskGroup group(rt);
    auto a = group.async([] { return 20; });
    auto b = group.async([] { return 22; });
    group.wait();
    return a.get() + b.get();
});
assert(parent.get() == 42);
```

显式 wait 才能在局部捕获异常

```
C++20
auto recovered = executor.async([](tf1::Runtime& rt) {
    try {
        tf1::TaskGroup group(rt);
        group.silent_async([] {
            throw std::runtime_error("child failed");
        });
        group.wait();
    } catch (const std::runtime_error&) {
        return true;
    }
    return false;
});
assert(recovered.get());
```

析构函数是 noexcept，只协作等待，不重抛组内异常。仅把 group 的作用域放进 try/catch、却不显式 wait，无法接收析构期间的子异常。

默认提交不继承父停止请求。需要继承时使用 `async<true>` / `silent_async<true>`，并保证关联上下文在句柄仍可能查询继承停止状态期间有效。一般把组内 Future 的使用限制在组作用域内最清晰。

来源：core/task_group.hpp; examples/26_task_group.cpp; examples/16_error_handling.cpp。

15 / SubFlow 动态子图

SubFlow 同时提供 FlowBuilder 的构图能力和 Context 的执行期上下文。它用于依据运行时数据创建新的节点和边，而不是直接提交互相独立的 callable。

```
C++20
int result = 0;
auto parent = executor.async([&](tf1::SubFlow& sf) {
    auto a = sf.emplace([&] { result = 21; });
    auto b = sf.emplace([&] { result *= 2; });
    a.precede(b);
    sf.run();
    sf.wait();
});
parent.get();
assert(result == 42);
```

run 与 wait 分别负责什么

run 提交子图源节点并立即返回；wait 协作等待挂接在当前父节点上的子工作。只调用 emplace 后返回不会自动执行子图。已经 run 的子图未完成前，不要再次提交或编辑它。

同一个 SubFlow 节点再次执行时，内部 Graph 会清空并重建。上一轮生成的 Task 句柄不能跨轮次保存使用。SubFlow& 本身也不能逃逸出当前回调。

与其他方式的选择

已知信息	使用方式
提交前就有完整子图	Flow 模块嵌套
执行期间决定若干独立子任务	Runtime::async 或 TaskGroup::async
执行期间决定新的依赖结构	SubFlow 构图后 run

如果子图捕获父 callable 内的局部变量，务必在变量销毁前显式 wait。需要局部截获子图异常时，可在提交前建立 ScopedExceptionAnchor，并在其有效范围内完成等待，见第 21 节。

来源：core/subflow.hpp; examples/27_dynamic_subflow.cpp; core/work_invokers.hpp。

16 / Branch 与 MultiBranch

Branch 选择单个后继，MultiBranch 选择多个后继。后继索引从 0 开始，对应当前边表顺序；初次用 precede(a, b) 建边时，a 为 0，b 为 1。修改边后应重新核对索引。

```
C++20
tfl::Flow flow;
int value = 0;
auto route = flow.emplace([](tfl::Branch& branch) {
    branch.select(0);
});
auto yes = flow.emplace([&] { value = 42; });
auto no = flow.emplace([&] { value = -1; });
route.precede(yes, no);
executor.async(flow).get();
assert(value == 42);
```

选择操作	语义
select(index) / operator()(index)	按位置选择
branch[index] = true / false	通过代理选择或取消
select_if(predicate)	以 TaskView 检查后继；单分支选首个匹配
unselect / reset	取消选择
MultiBranch::select_all	选择全部后继

MultiBranch 的 select_if 可以选择全部匹配项。单分支后一次选择覆盖前一次；未选择时本次不传播到后继。无效索引不会替应用建立一个“默认分支”，应在业务层明确处理。

汇聚时的常见误区

Branch 仍参与后继的普通依赖计数。若两个互斥路径都通过普通边连到同一个汇总节点，而每次只执行其中一条，汇总节点不能自然满足“两个前驱都到达”的条件。应按实际语义设计分支内后续流程，或使用经过验证的控制流模式。

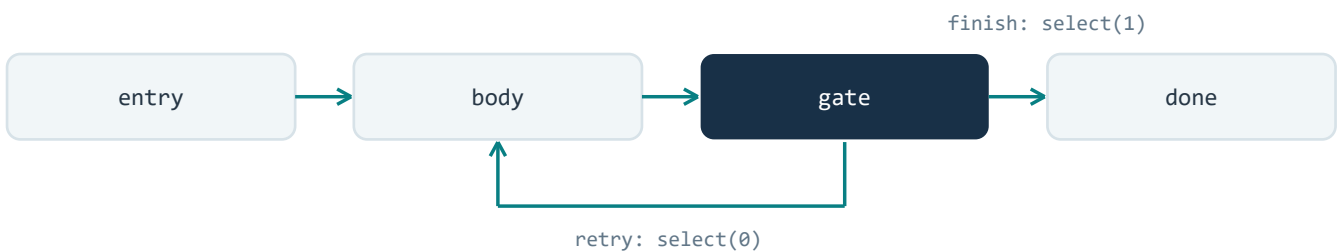
多个选中后继可能并行，写共享变量仍需要同步。不要把“条件选择”理解为自动串行。

来源：core/branch.hpp；core/executor.hpp；examples/05_branch.cpp。

17 / Jump、MultiJump 与循环

Jump 强制激活一个选定后继，MultiJump 强制激活多个后继。它们绕过目标的普通依赖屏障，适用于显式状态机和循环；这种能力也要求调用方避免把正在运行的目标再次激活。

```
C++20
tfl::Flow flow;
int attempts = 0;
bool finished = false;
auto entry = flow.placeholder();
auto body = flow.emplace([&] { ++attempts; });
auto gate = flow.emplace([&](tfl::Jump& jump) {
    jump.select(attempts < 3 ? 0 : 1);
});
auto done = flow.emplace([&] { finished = true; });
entry.precede(body);
body.precede(gate);
gate.precede(body, done);
executor.async(flow).get();
assert(attempts == 3 && finished);
```



与普通分支的区别

Branch 的目标仍等待普通前驱条件；Jump 直接激活目标。普通节点形成严格环会被默认建边检查拒绝，含 Jump 的受控路径使用另一套调度语义。

按当前实现，Jump 未选择目标或 reset 后，本次不会自动执行全部后继。上例在结束时显式 select(1)，使 done 真正被执行。不要根据旧示例的日志名称推断有自动“落到下一条”的行为。

MultiJump 增加并行激活目标的能力；不要让多个分支同时跳回同一个尚未结束的节点。设计循环时要写明入口、重复条件、出口和共享状态同步方式，并为停止条件设置明确边界。

来源：core/jump.hpp；core/executor.hpp::tear_down_jump_task；examples/06_jump.cpp。

18 / TaskObject 与 AsyncTaskObject

对象任务把业务对象直接构造在任务存储中，不需要先建立一个可复制或可移动的临时 callable。
`TaskObject<T>` 是图节点句柄；`AsyncTaskObject<R, T>` 同时保留对象访问和异步结果访问。

```
C++20
struct Counter {
    explicit Counter(int initial) : value(initial) {}
    Counter(const Counter&) = delete;
    Counter& operator=(const Counter&) = delete;
    int operator()() { return value += 2; }
    int value;
};

tf1::Flow flow;
auto node = flow.emplace_object<Counter>(40);
executor.async(flow).get();
assert(node.object().value == 42);

auto task = executor.defer_async_object<Counter>(40);
task.start();
assert(task.get() == 42);
assert(task.object().value == 42);
```

object 与 get 分工

`object` 返回实际业务对象；`get` 返回异步执行结果，二者可能类型不同。对象的 `operator()` 也可接受 `Runtime&` 或 `SubFlow&`。自定义 `graph_holder` 可作为模块对象，并支持构造参数 `tuple`、次数和谓词组合。

先保存对象任务的句柄，再执行链式配置和 `start`。继承的链式接口可能返回基类引用，链式表达式的静态类型不一定继续保留 `object` 入口。

`object` 不等待，也不加锁；即使句柄是 `const`，当前接口仍采用指针式访问语义。启动前配置或完成后读取最直接，运行中访问需要应用同步。`TaskObject` 不延长图寿命；`AsyncTaskObject` 的对象引用也不能脱离底层任务存活期使用。

来源：core/task_object.hpp; core/async_task_object.hpp; examples/31_task_object.cpp; examples/32_async_task_object.cpp。

19 / Semaphore 与资源配额

Semaphore 在任务实际执行前申请配额，完成后按配置释放配额。获取不到资源的 Work 进入等待路径，而不是让用户 callable 占着 Worker 一直阻塞。它常用于连接池、并发 I/O 名额或多资源需求。

```
C++20
tfl::Semaphore slots(2);
tfl::Flow flow;
std::atomic<int> completed{0};
for (int i = 0; i < 8; ++i) {
    auto task = flow.emplace([&] { ++completed; });
    task.acquire(slots).release(slots);
}
executor.async(flow).get();
assert(completed.load() == 8);
assert(slots.value() == 2);
```

配置	含义
Semaphore(max)	初始可用配额等于上限
Semaphore(max, current)	指定初始可用值，可从 0 开始
acquire(sem, count)	任务执行前一次申请多个单位
release(sem, count)	执行收尾归还或提供配额
clear_acquires / clear_releases	静止状态下移除配置
reset(max, current)	无待处理等待者等允许条件下重设容量

正确配置

配额请求不能超过可能提供的容量；release 不应使可用值超过上限。限流通常成对 acquire / release；事件通知可以由生产者 release、消费者只 acquire，但须严格核算总量。

当前多信号量协议按全局指针顺序持有目标锁，先检查全部请求，再统一扣减。失败时不先扣掉一部分配额。因此 acquire 遍历不保证添加顺序；测试应按信号量身份匹配 count，而不能断言 got[0] 对应第一个添加对象。

来源：core/semaphore.hpp；core/work.hpp::_try_acquire_semaphores；examples/07_semaphore.cpp。

20 / 异常传播与等待接口

任务抛出的普通 C++ 异常由框架保存，之后经相应结果或显式等待接口交给调用方。配置阶段的参数错误和分配失败则可能直接从提交函数抛出，不应只围住 `get`。

```
C++20
auto failed = executor.async([]() -> int {
    throw std::runtime_error("calculation failed");
});
failed.wait();
bool caught = false;
try {
    (void)failed.get();
} catch (const std::runtime_error&) {
    caught = true;
}
assert(caught);
```

位置	推荐接收方式
普通 <code>async</code> 任务	返回 <code>Future</code> 的 <code>get</code>
静态图内部节点	图提交返回的 <code>Future::get</code>
<code>TaskGroup</code> 子工作	回调内部显式 <code>group.wait</code> 并捕获
顶层 <code>silent_async</code>	<code>callable</code> 内部自行捕获和报告
依赖的异步前驱	后继内显式 <code>predecessor.get</code>

失败节点的普通依赖后继不会按正常路径继续传播；已经开始运行的独立工作不会回滚副作用。异常处理应区分“阻止后续工作”和“撤销外部写入”。后者要由应用通过事务或补偿逻辑实现。

`tfl::Exception` 用于库层面的接口错误，普通用户异常保留自身类型。观察者和 `WorkerHandler` 回调是 `noexcept`，不能把错误直接抛出回调，否则会终止程序。

反复调用 `Future::get` 可能再次重抛保存的异常。用 `wait` 忽略异常只代表你没有读取错误，不表示任务成功。

来源：core/exception.hpp; core/async_future.hpp; examples/16_error_handling.cpp; examples/34_dependency_errors.cpp。

21 / ScopedExceptionAnchor

ScopedExceptionAnchor 在当前 Context 上建立一个词法作用域异常锚点，使子任务异常先归档到当前 Work。等待接口随后重抛，调用者可在本层恢复，避免错误直接沿默认路径继续上行。

使用时显式包含 <taskflowlite/core/scoped_exception_anchor.hpp>；当前主入口头文件没有提供该类型的完整定义。

```
C++20
auto parent = executor.async([](tf1::SubFlow& sf) {
    bool caught = false;
    try {
        tf1::ScopedExceptionAnchor anchor(sf);
        (void)sf.emplace([] {
            throw std::runtime_error("subgraph failed");
        });
        sf.run();
        sf.wait();
    } catch (const std::runtime_error&) {
        caught = true;
    }
    return caught;
});
assert(parent.get());
```

生效时间与嵌套

必须在启动需要拦截的子任务之前构造锚点，并保留到这些子任务完成。只在异常发生后临时构造，不能追溯拦截已经传播的错误。

同一个 Work 上的锚点应严格嵌套。内部只有第一次设置标志的对象取得清理权，内层对象析构不会提前移除外层锚点。不能跨线程重叠使用，也不能让 Context 比锚点先失效。

TaskGroup 已有自己的组级异常管理。一般优先用 TaskGroup 表达一组独立子工作；需要为现有 Runtime / SubFlow 建立明确恢复边界时，再使用显式锚点。

异常锚点不是线程同步原语，也不会隔离业务内存或撤销已完成的副作用。局部 catch 成功后，是否允许后续业务继续运行，仍取决于应用状态是否一致。

来源：core/scoped_exception_anchor.hpp; core/runtime.hpp; core/subflow.hpp。

22 / 协作取消与停止请求

`request_stop` 发出停止请求，`stop_requested` 用于观察请求。调度器在适当边界检查状态，但不会强制中断已经进入 callable 的线程；长任务应自行设置检查点。

```
C++20
tfl::AsyncTask<void> task;
std::atomic<bool> entered{false};
task = executor.defer_async([&] {
    entered.store(true, std::memory_order_release);
    while (!task.stop_requested()) {
        std::this_thread::yield();
    }
});
task.start();
while (!entered.load(std::memory_order_acquire)) {
    std::this_thread::yield();
}
task.request_stop();
task.get();
```

此片段还需包含 `<thread>`。循环仅演示协议，实际任务应在合理粒度的计算或业务步骤之间检查，而不是采用持续空转。

停止域

Runtime 和 TaskGroup 提交默认 `InheritTopology=false`；需要父停止请求传到子任务时显式使用 `async<true>` / `silent_async<true>`。这个模板参数控制 Topology 父链，不移除 Work 的父子完成计数，也不等于异常隔离。

继承停止状态会引入父上下文生命周期要求。不要让子句柄在父或组已经销毁后继续查询依赖父链的状态。独立 `defer_async` 任务是自己的顶层停止域。

业务上的取消

停止请求没有保证某个循环恰好执行多少次，也没有自动回滚。取消后仍应等待工作结束，再释放输入、图、信号量或外部句柄。对不可中断的阻塞调用，应使用该资源本身的超时或取消机制。

来源：core/async_future.hpp; core/topology.hpp; examples/17_cancellation.cpp。

23 / TaskObserver 任务观察者

TaskObserver 通过 on_before 和 on_after 观察 callable 的执行。观察者注册在任务上，由共享所有权管理；回调在执行该任务的 Worker 上发生。重复执行的任务会产生多轮事件。

```
C++20
struct Counts : tfl::TaskObserver {
    std::atomic<int> before{0}, after{0};
    void on_before(tfl::WorkerView) noexcept override {
        ++before;
    }
    void on_after(tfl::WorkerView) noexcept override {
        ++after;
    }
};

tfl::Flow flow;
auto task = flow.emplace([] {});
auto counts = task.register_observer<Counts>();
executor.async(flow, 3ULL).get();
assert(counts->before.load() == 3);
assert(counts->after.load() == 3);
```

并发与开销

同一个观察者可由不同 Worker 并发调用，成员容器和计数需要同步。回调必须 noexcept；在里面分配内存、写日志或调用可能失败的 API 时，应自行处理异常。

任务执行时间很短时，日志、锁和时间戳的开销可能显著改变性能。可使用每线程缓冲，执行完再汇总；不要在热路径同步刷新文件。

WorkerView 可查询 worker id、队列快照和线程信息，它不是持久执行快照。不要保存它到观察回调之外，也不要 queue_size 推断任务完成。

测量多个并行任务后，单个任务耗时之和通常大于端到端墙钟时间；两者不能直接互换。完整计时和追踪示例见 15、21。

来源：core/observer.hpp; core/task.hpp; examples/15_observer.cpp; examples/21_observer_tracing.cpp。

24 / Worker、WorkerHandler 与 Context

Worker 是执行器内部工作线程对象；Context 暴露当前 worker 和 executor；WorkerView 是只读的短期观察视图。应用无需自行创建或接管 Worker 的调度循环。

```
C++20
struct Lifecycle : tfl::WorkerHandler {
    std::atomic<int> starts{0}, stops{0};
    void on_start(tfl::Worker&) noexcept override {
        ++starts;
    }
    void on_stop(tfl::Worker&) noexcept override {
        ++stops;
    }
};
Lifecycle handler;
{
    tfl::Executor local(handler, 2);
    auto id = local.async([](tfl::Runtime& rt) {
        return rt.worker().id();
    });
    assert(id.get() < 2);
}
assert(handler.starts.load() == 2);
assert(handler.stops.load() == 2);
```

回调的用途和契约

on_start / on_stop 可用于线程相关初始化和清理。多个线程可能同时调用同一个 handler，因此共享状态需要同步。两个回调均为 noexcept；on_stop 当前只有 Worker& 参数，不接受 exception_ptr。

handler 是借用对象，必须在 Executor 析构完成后才能销毁。上例把 handler 放在 executor 外层，利用 C++ 作用域顺序表达这个关系。

Context::worker / executor 返回的是关联对象引用。不能在其他线程使用回调内 Worker 上下文，也不要对库管理的线程执行 join、detach 或重置线程对象。线程生命周期由 Executor 统一管理。

来源：core/worker.hpp；core/context.hpp；examples/30_worker_handler.cpp。

25 / 图导出、命名与 D2

Flow::dump 将任务图导出为 D2 文本，支持字符串返回和输出流形式。D2Renderer 把节点类型、依赖、嵌套图和信号量标注组织成可视化描述；外部 D2 工具负责布局与渲染。

```
C++20
tfl::Flow flow;
flow.name("report_pipeline");
auto read = flow.emplace([] {} ).name("read");
auto compute = flow.emplace([] {} ).name("compute");
auto save = flow.emplace([] {} ).name("save");
flow.linearize(read, compute, save);
std::string d2 = flow.dump(tfl::Direction::Right);
assert(d2.find("report_pipeline") != std::string::npos);
```

保存和渲染示意：

```
DISPLAY SNIPPET
std::ofstream output("workflow.d2");
flow.dump(output, tfl::Direction::Right);
```

```
COMMAND
d2 workflow.d2 workflow.svg
```

观察结构，不等于观察执行历史

图导出展示配置关系，不是执行时间线，也不证明某条条件路径实际执行过。动态 SubFlow 只有运行时才知道具体节点；要分析实际事件和耗时，应配合 TaskObserver。

Direction 支持 Down、Right、Up、Left，默认 Down。给关键节点命名比依赖自动地址标识更利于排错。导出时不要并发修改图、节点名称或资源配置。

完整静态图示例在 examples/10_dump.cpp；仓库已有 documentation/img/d2.svg 可作阅读示范。D2 是额外渲染工具，不是库执行所需依赖。

来源：core/flow.hpp；core/d2_render.hpp；examples/10_dump.cpp。

26 / 常见并行模式

任务图是表达方式，性能仍取决于任务粒度、数据布局和依赖结构。先选择模式，再选择多少节点和多少线程。

模式	表达方法	实践重点
扇出与汇聚	多个工作节点，单个汇总后继	汇总只在全部输入完成后读数据
分块并行	每个任务处理连续区间	减少极小任务和相邻缓存行争用
归约树	分层两两汇总	避免所有工作争同一把锁
流水线	Flow 中按阶段建边	跨批次重用缓冲时明确所有权
波前图	每个单元依赖已完成邻居	只表达必要边，保留可并行区域
递归分解	Runtime 派发更小的工作	小规模转为直接计算，限制调度开销

```
C++20
std::array<int, 4> input{1, 2, 3, 4};
std::array<int, 4> squares{};
int total = 0;
tfl::Flow flow;
auto reduce = flow.emplace([&] {
    total = std::accumulate(squares.begin(),
                           squares.end(), 0);
});
for (std::size_t i = 0; i < input.size(); ++i) {
    auto map = flow.emplace([&, i] {
        squares[i] = input[i] * input[i];
    });
    map.precede(reduce);
}
executor.async(flow).get();
assert(total == 30);
```

循环索引 `i` 按值捕获，各任务写不同元素；`reduce` 通过边等待所有结果。实际批量数据通常应按区间分块，避免一个标量对应一个任务。

仓库示例 09、18 的流水线通过 Flow 组合；`core/pipeline.hpp` 当前是注释草稿，没有可用的独立 Pipeline API。不要把草稿里的端口和数据流类型当成现有功能。

来源：examples/09_pipeline.cpp; 13_parallel_reduce.cpp; 18_pipeline_producer_consumer.cpp; 20_parallel_for_index.cpp。

27 / 完整案例：受限资源的并行批处理

下面是可以单独保存、编译和运行的完整程序。四个任务分别处理一个输入，信号量把业务并发上限设为 2；汇总节点在全部计算完成后运行，主线程通过 `get` 同步并接收异常。

```
C++20

#include <taskflowlite/taskflowlite.hpp>
#include <array>
#include <iostream>
#include <numeric>

int main() {
    tf1::Executor executor(4);
    tf1::Semaphore resource(2);
    tf1::Flow flow;
    std::array<int, 4> inputs{1, 2, 3, 4};
    std::array<int, 4> outputs{};
    int total = 0;

    auto finish = flow.emplace([&] {
        total = std::accumulate(outputs.begin(),
                                outputs.end(), 0);
    });
    finish.name("sum");
    for (std::size_t i = 0; i < inputs.size(); ++i) {
        auto task = flow.emplace([&, i] {
            outputs[i] = inputs[i] * inputs[i];
        });
        task.acquire(resource).release(resource);
        task.precede(finish);
    }
    try {
        executor.async(flow).get();
    } catch (const std::exception& error) {
        std::cerr << error.what() << '\n';
        return 1;
    }
    std::cout << total << '\n';
    return total == 30 ? 0 : 1;
}
```

扩展这个案例

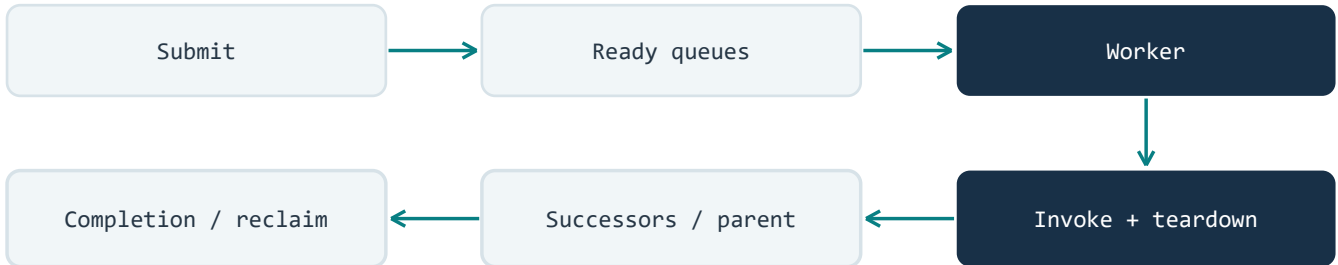
把平方计算替换为业务逻辑时，仍需保证每个输出槽只有一个写者。若任务写数据库或文件，失败后不会自动撤销之前任务的写入；可先写独立暂存结果，成功汇总后再提交业务结果。

当输入规模变化时，可以执行前重新构图，或在 `SubFlow` 内动态创建节点。图正在运行期间不能修改现有节点数量和边。验证结果后再测量线程数、分块尺寸和资源容量的影响。

来源：第 05、19、20、26 节；完整程序为本手册组合示例。

28 / 内部调度：从提交到完成

本节开始解释实现结构。应用无需调用这些内部函数，但理解它们有助于判断等待范围、任务粒度和并发约束。



无可运行任务时进入 Notifier；有新工作时发布并通知。

一次执行的主要阶段

1. 工厂根据 callable 协议创建 Work、Invoker 和必要的结果存储；静态图节点归 Graph，异步节点使用引用管理。
2. 提交建立 Topology 或父子关系，初始化依赖和活动工作计数，再发布可运行节点。
3. Worker 从本地队列、其他 Worker 或共享调度栈取得工作；没有工作时进入通知器的等待协议。
4. 执行前处理停止、资源配额和观察回调，然后调用具体 Invoker。
5. 收尾归档异常、释放配置的配额、通知依赖后继，并递减父任务或顶层活动计数。
6. 最终发布完成状态，唤醒等待者；没有引用的异步节点进入回收。

调度顺序不是业务契约

工作窃取让空闲线程帮助忙线程；本地执行常利用缓存与就近工作，任务就绪时也可能直接接续执行。不能据此推断任务固定在线程上运行、固定 FIFO，或同批任务按创建顺序完成。

内存可见性依赖发布、获取、计数和完成状态的同步协议。随意把原子内存序改弱、把计数调整挪到发布之后，可能使某些小测试通过却破坏并发交错。维护调度代码时需要针对具体交错验证。

来源：core/executor.hpp；core/worker.hpp；core/work_invokers.hpp。

29 / Work、Graph、Topology 与 ResultSlot

内部组件	主要职责
Graph	拥有静态图节点并提供图级存储与遍历
Work	节点载荷、边表、父关系、资源和执行控制
Topology	一次异步执行的状态、引用和停止关联
ResultSlot<R>	区分值、引用、void 的结果存储
WorkFactory	按 callable 协议构造匹配的 Work 与结果
Invoker	将具体 callable 与统一执行协议连接
Payload / 存储辅助	小对象内联与需要时的堆存储

两条“父关系”

Work 的父子关系参与子任务完成计数和异常锚点传播；Topology 的父链用于关联停止域等控制状态。这两条关系有不同职责。把 InheritTopology 设为 false，不会取消 Work 的父子完成关系。

边表布局

当前 Work 的统一边表把后继放在前缀，前驱放在后缀；m_num_successors 区分两段。静态图邻接关系是借用关系；异步前驱段中的每一项持有对应前驱引用。

插入、删除和动态登记必须保留这项分区不变量。删除可能通过交换末尾元素填补位置，因此“边的位置”不适合作为长期业务 ID。

完成和回收不同步骤

完成状态表明业务执行结束，但结果、callable 和前驱引用可能继续由外部句柄保留。ResultSlot 本身不拥有引用类型的外部对象。无外部句柄也不代表运行中 Work 可以立即销毁，执行引用仍负责保持有效。

Payload 的配置宏会影响 Work 布局；跨翻译单元使用不同宏可能产生不一致定义。不要通过直接改内部存储大小解决尚未测量的性能问题。

来源：core/work.hpp; core/graph.hpp; core/topology.hpp; core/result_slot.hpp; core/work_factory.hpp; core/work_storage.hpp。

30 / BoundedQueue 与 SharedWorkStack

BoundedQueue 是单 Owner、多 Stealer 的固定容量指针队列。Owner 从尾部 push / pop，其他线程从头部 steal。容量必须大于 1 且为二次幂；它不拥有指针对象，也不允许把 nullptr 当作有效工作入队。

C++20

```
tfl::BoundedQueue<int*, 2> queue;
int a = 1, b = 2, c = 3;
int rejected = 0;
auto overflow = [&](int*) { ++rejected; };
queue.push(&a, overflow);
queue.push(&b, overflow);
queue.push(&c, overflow);
assert(rejected == 1);
assert(queue.pop() == &b);
assert(queue.steal() == &a);
```

溢出是显式协议

当前 push 接收溢出回调，单元素回调得到未入队指针；批量回调得到剩余区间的迭代器和数量。没有旧式 try_push API。批量发布了前缀后，如果溢出回调抛异常，已经发布的部分不能整批重试。

SharedWorkStack 为共享调度提供非拥有 Work* 链。多个生产者原子发布 incoming 链，消费端取得独占权后接管和摘取节点。它不保证全局 FIFO 或严格 LIFO，也不保证整体消费过程 lock-free。

约束	影响
steal 返回空可能只是竞争失败	不能据此宣告系统无任务
size / empty 是近似快照	不能用来代替完成同步
Work::m_next 是侵入式链接槽	同一节点不能同时在多条运行期链
容器不拥有 Work	销毁或重复发布必须由外部协议控制

来源：core/bounded_queue.hpp; core/shared_work_stack.hpp; examples/35_bounded_queue_overflow.cpp。

31 / Notifier、SpinMutex 与随机选择

Notifier 负责在没有可运行工作时让 Worker 休眠，并在有工作时唤醒。它解决的是“准备睡眠和新工作发布可能同时发生”的协议问题，不能简单替换成先检查空、再无条件睡眠。

等待协议

PROTOCOL

```
prepare_wait(worker_id)
-> 再次检查是否有工作
-> 有工作: cancel_wait(worker_id)
-> 无工作: commit_wait(worker_id)
```

发布者在发布可运行工作后通知等待者。

prepare / commit / cancel 配合等待者状态和 epoch，避免通知恰好发生在入睡前而丢失。notify_one、notify_n、notify_all 选择唤醒范围。应用层应使用 Future 或上下文等待，不直接干预 Executor 内部通知器。

SpinMutex

SpinMutex 使用 atomic_flag，竞争时先进行有界增长的轮询，再 yield。它满足 lock、try_lock、unlock 形式，可与标准 RAII 锁配合。它适用于极短临界区，不提供公平性，也不是递归锁；同线程重复 lock 会永久等待。

SplitMix64

SplitMix64 提供轻量伪随机序列，用于工作窃取的目标选择。它保存局部状态，不是密码学随机数生成器。固定种子有利于复现实验，但多线程调度本身仍有不确定性。

维护时的重点

队列、通知器和调度循环构成完整协议。改其中一个模块时应检查“发布后通知”“睡前复查”“竞争失败重试”是否仍衔接。单次压力测试没有出现挂起，不能代替对丢唤醒交错的分析。

来源：core/notifier.hpp; core/spin_mutex.hpp; core/random.hpp; core/executor.hpp。

32 / SmallVector、ObjectPool 与存储

SmallVector 为少量元素提供内联存储，超出容量后使用堆存储。它被边表和控制流目标列表等结构使用，用于减少小容器的分配次数。

```
C++20
tfl::SmallVector<int, 4> values;
values.push_back(20);
values.push_back(22);
assert(values.size() == 2);
assert(values[0] + values[1] == 42);
```

SmallVector 的使用注意

增长、插入、移动或切换存储可能使迭代器和引用失效。即使某次操作没有分配，也不应把当前布局当作长期保证。元素构造和赋值可能抛异常；异常保证取决于具体操作和元素类型。容器不是线程安全的共享队列。

ObjectPool

ObjectPool 为固定类型对象批量分配 slab，并通过分桶和带标记的空闲链复用槽位。Work 对象池减少频繁分配节点的成本；槽位复用不等于旧对象生命周期继续存在。

带版本标记的指针用于处理空闲链中的 ABA 风险。底层原子操作是否真正 lock-free 取决于平台实现，不能因为源码使用 atomic 就宣称整个对象池无锁。

Payload 和结果

小型 Invoker 可以放进 Work 的内联 Payload，超过容量或对齐要求时使用堆存储。对象任务直接在这套存储中构造业务对象；结果由 ResultSlot 管理，二者职责不同。

排查内存问题时可关闭 TFL_ENABLE_TASK_POOL 以获得更直接的分配和释放轨迹。修改 TFL_WORK_PAYLOAD_SIZE 会改变布局和每个节点的成本，应基于真实 callable 尺寸测量，并确保所有翻译单元配置一致。

来源：core/small_vector.hpp; core/object_pool.hpp; core/work_storage.hpp; core/result_slot.hpp。

33 / Concepts、工具类型、枚举与宏

组件	作用
basic_invocable 等 concepts	决定 callable 对应的任务协议
graph_holder	接受 Flow 或其他符合持图约定的对象
async_future / async_task	识别结果与延迟任务句柄
predicate / callback / capturable	约束重复谓词、完成回调与存储构造
pack / tuple 支持	组织批量构造及模块构造参数
Immovable / MoveOnly	表达稳定地址或独占所有权要求
Located / Exception	将诊断信息与源码位置关联
TaskType / Direction / Version	类型标识、图布局方向和源码版本

TaskType 包括 Placeholder、Basic、Branch、MultiBranch、Jump、MultiJump、Runtime 和 Graph。Graph 类型可能表示模块或子图相关工作，不能只看名字推断具体业务对象。

影响诊断和布局的宏

宏	当前默认或作用
TFL_ENABLE_ASSERT	Debug 开启；Release 默认关闭
TFL_ENABLE_WORK_EXECUTION_CHECK	Debug 开启；检测 Work 执行生命周期重入
TFL_ENABLE_TASK_POOL	默认 1，控制 Work 对象池
TFL_WORK_PAYLOAD_SIZE	默认 128 字节，影响内联载荷与 ABI
TFL_DEFAULT_QUEUE_SIZE	默认 1024，本地队列容量
TFL_CACHE_LINE_SIZE	按目标平台选择，用于缓存行隔离

同一程序的相关翻译单元必须一致配置布局和行为宏。通过 CMake target_compile_definitions 统一传递比在不同 cpp 里零散 define 更稳妥。

注意：当前 Release 手动开启 TFL_ENABLE_ASSERT 后使用编译器假设，不是普通运行期诊断；违反假设会导致未定义行为。调试错误应优先使用 Debug、执行一致性检查和合适的 sanitizer。

来源：core/traits.hpp；core/utility.hpp；core/enums.hpp；core/macros.hpp；taskflowlite.hpp。

34 / 构建选项、依赖与预设

CMake 选项	默认值	用途
TFL_BUILD_EXAMPLES	顶层 ON，子项目 OFF	构建可运行示例
TFL_BUILD_TESTS	OFF	构建并注册测试
TFL_TEST_HEADERS	ON	启用测试时检查头文件独立编译
TFL_BUILD_BENCHMARKS	OFF	构建两套对比程序
TFL_BUILD_DOCS	OFF	提供 GenerateDocs 目标
TFL_BUILD_CORE_REPOS	OFF	构建已知问题的隔离复现
TFL_SANITIZER	OFF	OFF、ASAN 或 TSAN
TFL_NATIVE_ARCH	OFF	GCC/Clang 内部 Release 使用本机优化

常用预设

COMMAND

```
cmake --preset release
cmake --build --preset release --parallel 4
ctest --preset release -LE perfile --no-tests=error
```

release 使用 Ninja 并启用测试。Visual Studio 使用 windows-release；asan、tsan 和 windows-asan 用于对应检查。Windows 不支持这里的 TSAN 配置。Ninja 在 Windows 上需要可用的编译器开发环境。

可选依赖

测试默认固定 Catch2 v3.15.0 的 amalgamated 文件并校验哈希；基准对比固定 Taskflow 的版本提交。离线时分别设置 TFL_CATCH2_LOCAL_PATH 和 TASKFLOW_LOCAL_PATH。显式路径错误会失败，不会自动回退下载。

镜像和代理由 TFL_GITHUB_MIRROR、TFL_GITHUB_PREFIX、TFL_GIT_PROXY 控制。切换依赖版本时应同步对应哈希或完整提交；不要把随机变化的依赖分支用于可重复性能比较。

来源：CMakeLists.txt；CMakePresets.json；cmake/README.md；cmake/TflDependencies.cmake。

35 / 测试、CI 与文档生成

在完整检出的仓库根目录执行：

COMMAND

```
cmake -S . -B build/tests -DCMAKE_BUILD_TYPE=Release -DTFL_BUILD_TESTS=ON
cmake --build build/tests --config Release --parallel 4
ctest --test-dir build/tests -C Release --output-on-failure -LE perfile --no-tests=error
```

perfile 标签用于按文件重复注册的测试；排除它可以避免重复跑一遍同类用例，与平台 CI 一致。
--no-tests=error 防止“没有任何测试”被误认为成功。

检查层次

- 普通单元测试验证公开接口、依赖、对象生命周期和结果。
- 可运行示例验证文档场景；它们也能暴露接口变更后失效的调用。
- 逐头文件编译检查避免接口偶然依赖其他头文件提前包含。
- ASan 关注内存访问错误；TSan 关注可检测的数据竞争。通过并不构成对所有交错的证明。
- 独立故障复现检查特殊失败路径，应保留真实失败，不混进“全部通过”的宣传数字。

当前工作区的实际限制

本手册制作时，工作区多数 test/ 文件已处于删除状态，仅恢复了前一轮修复的 test_task.cpp。因此上面的完整测试命令描述完整仓库的使用方式，本次不能据此声称已重新运行整套项目测试。

本手册另行编译和运行其中的 C++ 示例，并在验证记录页说明结果。历史审查报告中的通过数量只代表当时运行，不作为当前目录状态的替代证据。

生成 API 文档

COMMAND

```
cmake -S . -B build/docs -DTFL_BUILD_EXAMPLES=OFF -DTFL_BUILD_DOCS=ON
cmake --build build/docs --target GenerateDocs
```

需要安装 Doxygen。Windows ASan 的运行库路径准备参见 cmake/README.md 和 [github/scripts/asan_runtime.ps1](https://github.com/taskflowlite/scripts/asan_runtime.ps1)。

来源：[.github/workflows/](https://github.com/taskflowlite/workflows/)；test 的版本库配置；documentation/CMakeLists.txt。

36 / 常见问题与当前边界

现象	优先检查
std::format 或原子等待不可用	编译器和标准库是否匹配，是否启用 C++20
Worker 内等待挂起	是否阻塞等待同池未完成任务，能否改为协作等待
start 报前驱未启动	对 Idle 前驱先 start，再提交后继
SubFlow 没有执行	是否只构图却遗漏 run
互斥分支后的汇总不执行	是否误把未选择分支也作为必须满足的普通前驱
取消后任务仍运行一段时间	callable 是否设置停止检查点，是否阻塞在外部 API
看不到子异常	是否只调用 wait，或只依赖 TaskGroup 析构
信号量计数测试跨平台失败	是否错误依赖 acquire 的遍历顺序
结果引用失效	是否销毁最后句柄或被引用的原始对象
队列 size 为 0 但仍有任务	是否把调度快照当成完成同步

特殊失败路径

当前源码中，AsyncTask::start 的准备阶段在持有自身控制锁时 reserve；动态前驱登记也在持锁期间扩展边表。缺少完整异常回滚的路径意味着不能保证分配失败后任务仍可安全重试。手册核对了源码，但没有在本次再次做分配故障注入。

Task::work 新 callable 构造失败后不能依赖旧 callable 保留。详见第 06 节。对这些边界，业务程序应避免假设强异常保证，库维护者应以专门故障复现验证修复。

旧审查报告记录过多信号量逐项获取回滚风险；当前实现已经改为统一持锁检查、全部满足后扣减。该旧交错描述不能直接当成当前实现的已复现故障，也不能仅凭改动断言所有并发风险都已排除。

来源：core/async_task.hpp::start; core/executor.hpp::_link_predecessors; core/work.hpp; 历史审查报告。

37 / 性能测量与优化方法

先确认结果和同步正确，再分析任务划分。线程多并不必然更快；任务太小会让调度开销占主导，任务太大则可能留下无法分摊的长尾。

推荐测量顺序

1. 固定编译器、构建类型、依赖提交、线程数和工作负载，记录硬件与运行环境。
2. 先执行带正确性校验的 smoke，确认输入规模和输出都符合预期。
3. 测量端到端时间，再分离构图、提交、执行、同步与销毁成本。
4. 多次运行，报告分布或中位数，观察冷启动和稳态差异。
5. 只改变一个主要因素，如线程数、块大小或资源容量，再比较。

COMMAND

```
bench_taskflowlite --smoke
bench_taskflow --smoke
```

仓库基准的计时边界

两套基准使用同场景和校验逻辑。默认包含正确性原子计数操作，测得的不只是调度开销。TaskflowLite 当前在计时前 defer_async 创建任务，计时区间内 start().wait()，因此任务创建成本不包含在这段时间里。

--smoke 保留图结构，但把重复次数限制到很小；其结果用于验证可运行性，不用于性能结论。两套程序都使用 --no-verify 时会关闭计数校验，应先完成带校验的运行再比较。

常见优化方向

复用 Executor 和可复用图、按连续数据分块、减少共享原子热点、控制观察日志开销、避免大量互斥小任务。启用本机指令集优化会影响二进制可移植性；修改内联容量和对象池应观察内存占用与吞吐的共同变化。

README 中的性能表是历史数据，未记录精确双方提交和多次测量分布。它不能作为当前快照的性能保证，本手册不重新包装成新的基准结论。

来源：benchmarks/README.md；benchmarks/bench_taskflowlite.cpp；benchmarks/bench_taskflow.cpp。

38 / 组件与源码速查

本表覆盖主要用户组件和实现支撑模块。源码路径相对于 taskflowlite/core/；精确模板约束请查阅对应文件。

组件或问题	源码入口	手册章节
图所有权与构建	flow.hpp、flow_builder.hpp、graph.hpp	05、08
节点句柄与只读视图	task.hpp	06
执行器与提交	executor.hpp	09、28
结果和延迟任务	async_future.hpp、async_task.hpp	10、11、12
对象任务	task_object.hpp、async_task_object.hpp	18
动态上下文	context.hpp、runtime.hpp、subflow.hpp	13、15、24
分组与异常锚点	task_group.hpp、scoped_exception_anchor.hpp	14、21
分支和跳转	branch.hpp、jump.hpp	16、17
资源配额	semaphore.hpp、work.hpp	19
异常与停止	exception.hpp、topology.hpp	20、22
任务与线程观察	observer.hpp、worker.hpp	23、24
D2 渲染	d2_render.hpp	25
节点与执行协议	work.hpp、work_invokers.hpp	28、29
工厂、载荷与结果	work_factory.hpp、work_factory_fwd.hpp、work_storage.hpp、result_slot.hpp	29、32
工作队列与共享栈	bounded_queue.hpp、shared_work_stack.hpp	30
等待与短临界区	notifier.hpp、spin_mutex.hpp	31
窃取目标随机数	random.hpp	31
小容器与对象池	small_vector.hpp、object_pool.hpp	32
约束和辅助类型	traits.hpp、utility.hpp、forward.hpp	33
枚举、版本与宏	enums.hpp、macros.hpp、主入口头文件	01、33
Pipeline 草稿	pipeline.hpp，仅注释	26

内部声明存在于头文件中不代表它们是建议直接使用的稳定业务 API。优先通过主入口头文件和前半部分介绍的类型构建应用。旧 UnboundedQueue 已不在当前有效实现中。

来源：taskflowlite/taskflowlite.hpp；taskflowlite/core/ 文件集合。

39 / 全部示例阅读路线

以下编号对应 examples/ 下的独立 cpp 文件。先读基础，再按业务需要选择控制流、动态任务和观察功能。详细文件名可在 examples/README.md 查看。

编号	主题	编号	主题
01	基础 DAG	19	依赖异步任务
02	并行任务	20	并行索引处理
03	重复与循环	21	观察者追踪
04	Runtime	22	状态机
05	Branch	23	并行归约
06	Jump / MultiJump	24	递归 Runtime
07	Semaphore	25	重试与退避
08	静态子图模块	26	TaskGroup
09	Flow 流水线	27	动态 SubFlow
10	D2 图导出	28	任务编辑与重绑定
11	emplace 重载	29	Future 结果类型
12	循环工作流	30	WorkerHandler
13	并行归约	31	TaskObject
14	异步任务链	32	AsyncTaskObject
15	Observer	33	unchecked 建边
16	异常处理	34	依赖错误处理
17	协作取消	35	有界队列溢出
18	生产消费流水线		

构建示例

COMMAND

```
cmake -S . -B build/examples -DCMAKE_BUILD_TYPE=Release -DTFL_BUILD_EXAMPLES=ON
cmake --build build/examples --config Release --target run_all_examples
```

只构建第一个示例时，目标名为 tfl_ex_01_basic_dag。示例可执行文件位于对应构建目录的 bin/examples 下，多配置生成器还会按 Release 等配置分目录。

若旧示例注释与当前实现冲突，应先核对接口和实际结果。手册中的 Jump 例子显式选择退出节点，就是为了避免把“没有选择目标”误读为自动走成功路径。

来源：examples/README.md；examples/CMakeLists.txt；examples/01 至 35。

40 / 来源、验证记录与后续维护

这份手册依据什么

主要依据当前工作区中的 taskflowlite/ 头文件、README、中英文用法、CMake 配置和 35 个示例。基线提交 91424ac 上叠加了未提交变更，因此本手册标为本地源码快照，不冒充某个已发布二进制的认证文档。

资料	用途
taskflowlite/taskflowlite.hpp 与 core/	版本、接口、调度和生命周期
CMakeLists.txt、CMakePresets.json、cmake/README.md	构建、集成、依赖与预设
examples/01 至 35	公开用法和组合场景
documentation/async-task-dependency-design.md	依赖、引用保留和失败边界
documentation/core-api-notes.md	接口变化与注释差异线索
documentation/core-review-2026-09-21.md	历史审查背景，需与现状重新核对
benchmarks/README.md	基准参数、校验和计时范围

项目主页：<https://github.com/wicyn/taskflowlite>

本次验证范围

本手册的 28 个可执行 C++ 代码块已在 Windows x64 / MSVC 19.44 / C++20 下编译并运行通过。完整程序检查退出码，片段保留结果断言；展示性片段、命令行和流程伪代码明确标注，未冒充独立程序。

PDF 检查包括中文字体嵌入、可提取文本、目录与书签、代码行宽、分页、页码和渲染后的逐页版面。验证结果以随手册生成的实际检查为准。

本次没有运行完整项目测试、GCC/Clang 或 macOS 验证，也没有重新进行分配故障注入、ASan/TSan 或性能测量。当前缺失测试文件的情况见第 35 节。

如何维护

本 PDF 对应同目录 TaskflowLite-Guide.zh-CN.md 源稿。公开接口或生命周期规则变化时，应同时修改示例、源稿和 README；内部实现变化时重点复核第 28 至 36 节。每次重制都应重新验证示例与版面，保留新的源码快照信息。

本手册说明库的行为和使用约定，项目代码及使用许可仍以仓库 LICENSE 为准。